Academy Hub — Sesión 5

CRUD completo de todas las apps, formularios, vistas, templates y permisos

---

Índice

1. Objetivo general de la sesión
2. Qué vamos a aprender
3. Teoría: qué significa CRUD
4. Teoría: Function Based Views vs Class Based Views
5. Vistas genéricas que utilizaremos
6. Convenciones del proyecto
7. Mapa de permisos por rol y entidad
8. CRUD de `core` — `ContactMessage`
9. CRUD de `accounts` — `User`
10. CRUD de `academies` — `Academy` y `Teacher`
11. CRUD de `courses` — `Category`, `Course`, `Convocatoria`, `CourseImage`
12. CRUD de `enrollments` — `Enrollment`, `WaitlistEntry`, `EnrollmentAnswer`, `EnrollmentDocument`
13. URLs finales por app
14. Tests transversales
15. Checklist global de la sesión
16. Commit recomendado
17. Registro de sesión
18. Resultado final

---

1. Objetivo general de la sesión

En la Sesión 4 construimos la estructura de datos completa de Academy Hub:

```
accounts      → User
academies     → Academy, Teacher
courses       → Category, Course, Convocatoria, CourseImage
enrollments   → Enrollment, WaitlistEntry, EnrollmentAnswer, EnrollmentDocument
core          → ContactMessage
```

Ahora vamos a convertir esos modelos en funcionalidad real, aplicando el patrón CRUD completo del PDF a **todas las apps**.

Al terminar esta sesión:

- Un visitante podrá ver listados y detalles públicos.
- Un alumno podrá gestionar sus inscripciones, reservas, documentos y lista de espera.
- Una academia podrá gestionar sus academias, profesores, cursos, convocatorias e imágenes.
- El staff podrá ver mensajes de contacto y gestionar usuarios y categorías.

Además, aplicaremos seguridad a nivel de objeto en todas las entidades:

> Un usuario solo podrá editar o eliminar aquello que le pertenece.

---

2. Qué vamos a aprender

- CRUD completo en todas las apps.
- `forms.py` por app.
- Vistas genéricas: `ListView`, `DetailView`, `CreateView`, `UpdateView`, `DeleteView`.
- `LoginRequiredMixin` y `UserPassesTestMixin`.
- Object-level permissions.
- Permisos por rol: alumno, academia, staff.
- `form_valid()`, `get_queryset()`, `get_absolute_url()`.
- `select_related` y `prefetch_related`.
- Paginación.
- Templates reutilizables.
- Tests de vistas y de permisos.
- Logging de acciones críticas.

---

3. Teoría: qué significa CRUD

```
C → Create
R → Read
U → Update
D → Delete
```

Aplicado a Academy Hub:

```
CREATE  → Publicar un curso
READ    → Ver el curso
UPDATE  → Editar el curso
DELETE  → Eliminar el curso
```

Lo mismo aplica a `Academy`, `Teacher`, `Convocatoria`, `Enrollment`, etc.

---

4. Teoría: Function Based Views vs Class Based Views

Hasta ahora hemos usado vistas como:

```
def home(request):
    ...
```

Esto se llama:

```
Function Based View
```

Django también ofrece:

```
Class Based Views
```

por ejemplo:

```
class CourseListView(ListView):
    ...
```

Son especialmente útiles para CRUD porque Django ya implementa gran parte del comportamiento repetitivo.

---

5. Vistas genéricas que utilizaremos

```
ListView      → lista
DetailView    → detalle
CreateView    → crear
UpdateView    → editar
DeleteView    → eliminar
```

---

6. Convenciones del proyecto

Para no repetir lo mismo en cada app, aquí van las reglas comunes.

6.1. Nombres de URL

Cada app define su propio `app_name` y sus URLs con nombres en inglés y en snake_case:

- `list`
- `detail`
- `create`

- `update`
- `delete`
- `mine`
- `approve`
- `reject`
- `cancel`
- `verify`
- `reject_document`

6.2. Rutas con ID + slug

Cuando una entidad tiene slug propio (Course, Academy, Category), las URLs públicas usan `pk` + `slug`:

```
/courses/<int:pk>/<slug:slug>/
```

Cuando no lo tiene (Enrollment, Convocatoria, etc.), solo `pk`:

```
/enrollments/<int:pk>/
```

6.3. `get_absolute_url()`

Cada modelo que se pueda visitar públicamente implementa `get_absolute_url()`.

Ejemplo en `Course`:

```
def get_absolute_url(self):
    from django.urls import reverse
    return reverse(
        "courses:detail",
        kwargs={
            "pk": self.pk,
            "slug": self.slug,
        },
    )
```

Ventaja: `CreateView` y `UpdateView` redirigen automáticamente sin tener que definir `get_success_url()`.

6.4. Optimización de querysets

- `select_related` para `ForeignKey` y `OneToOne`.
- `prefetch_related` para `OneToMany` y `ManyToMany`.

6.5. Permisos por rol

Rol	Cómo se detecta
Visitante	`request.user.is_authenticated == False`
Alumno	`request.user.is_authenticated and request.user.is_student`
Academia	`request.user.is_authenticated and request.user.is_academy`
Staff	`request.user.is_staff`
Superusuario	`request.user.is_superuser`

6.6. Mensajes y logging

Cada acción crítica muestra un mensaje al usuario con `messages` y registra un log con `logger.info`.

Ejemplo:

```
import logging
logger = logging.getLogger(__name__)

messages.success(self.request, "Curso publicado correctamente.")
logger.info("Course created: id=%s user=%s", self.object.pk, self.request.user.pk)
```

6.7. Estructura de carpetas

Cada app tiene:

```
<app>/
    forms.py
    urls.py
    views.py
    tests.py
```

Y `templates/<app>/` con los HTML correspondientes.

---

7. Mapa de permisos por rol y entidad

Entidad	Visitante	Alumno	Academia	Staff	Propietario
ContactMessage	crear	crear	crear	ver / editar / borrar	—
User (perfil público)	ver	ver	ver	ver	—
User (listado)	—	—	—	ver	—
Academy (listado)	ver	ver	ver	ver	—
Academy (detalle)	ver	ver	ver	ver	—
Academy (crear)	—	—	sí	—	—
Academy (editar)	—	—	—	—	sí
Academy (eliminar)	—	—	—	—	sí
Teacher (listado)	ver	ver	ver	ver	—
Teacher (detalle)	ver	ver	ver	ver	—
Teacher (crear)	—	—	sí	—	—
Teacher (editar)	—	—	—	—	sí (academia)

Entidad	Visitante	Alumno	Academia	Staff	Propietario
Teacher (eliminar)	—	—	—	—	sí (academia)
Category (listado)	ver	ver	ver	ver	—
Category (detalle)	ver	ver	ver	ver	—
Category (crear / editar / eliminar)	—	—	—	sí	—
Course (listado)	ver	ver	ver	ver	—
Course (detalle)	ver	ver	ver	ver	—
Course (crear)	—	—	sí	—	—
Course (editar / eliminar)	—	—	—	—	sí (academia)
Convocatoria (crear / editar / eliminar)	—	—	—	—	sí (academia del curso)
CourseImage (crear / editar / eliminar)	—	—	—	—	sí (academia del curso)
Enrollment (crear)	—	sí	—	—	—
Enrollment (listar propias)	—	sí	—	—	—
Enrollment (listar de mis cursos)	—	—	sí	—	—
Enrollment (aprobar / rechazar / matricular / dar de baja)	—	—	sí	—	sí (academia)
Enrollment (cancelar propia)	—	sí	—	—	—
WaitlistEntry (crear)	—	sí	—	—	—
WaitlistEntry (cancelar propia)	—	sí	—	—	—
WaitlistEntry (ver de mis cursos)	—	—	sí	—	—
WaitlistEntry (notificar / convertir)	—	—	sí	—	—
EnrollmentAnswer (crear / editar)	—	sí	—	—	—
EnrollmentAnswer (ver)	—	sí	sí	—	—
EnrollmentDocument (crear / eliminar)	—	sí	—	—	—
EnrollmentDocument (verificar / rechazar)	—	—	sí	—	—

---

8. CRUD de `core` — `ContactMessage`

8.1. Modelo

```
class ContactMessage(models.Model):
    name = models.CharField(max_length=150)
    email = models.EmailField()
    subject = models.CharField(max_length=200)
    message = models.TextField()
    is_read = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
```

8.2. Formulario

```
core/forms.py
```

```
from django import forms

from .models import ContactMessage
```

```
class ContactMessageForm(forms.ModelForm):

    class Meta:
        model = ContactMessage
        fields = [
            "name",
            "email",
            "subject",
            "message",
        ]
        widgets = {
            "message": forms.Textarea(attrs={"rows": 5}),
        }
```

8.3. Vistas

core/views.py

```
from django.contrib import messages
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    UserPassesTestMixin,
)
from django.urls import reverse_lazy
from django.views.generic import (
    CreateView,
    DeleteView,
    DetailView,
    ListView,
    UpdateView,
)

from .forms import ContactMessageForm
from .models import ContactMessage


class ContactMessageCreateView(CreateView):
    model = ContactMessage
    form_class = ContactMessageForm
    template_name = "core/contact_form.html"
    success_url = reverse_lazy("core:home")

    def form_valid(self, form):
        messages.success(
            self.request,
            "Mensaje enviado correctamente. Gracias por contactarnos.",
        )
        return super().form_valid(form)
```

```
class StaffRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):

    def test_func(self):
        return self.request.user.is_staff


class ContactMessageListView(StaffRequiredMixin, ListView):
    model = ContactMessage
    template_name = "core/contact_message_list.html"
    context_object_name = "messages_list"
    paginate_by = 20

    def get_queryset(self):
        return ContactMessage.objects.all().order_by("-created_at")


class ContactMessageDetailView(StaffRequiredMixin, DetailView):
    model = ContactMessage
    template_name = "core/contact_message_detail.html"
    context_object_name = "contact_message"


class ContactMessageUpdateView(StaffRequiredMixin, UpdateView):
    model = ContactMessage
    fields = ["is_read"]
    template_name = "core/contact_message_form.html"

    def get_success_url(self):
        return reverse_lazy(
            "core:contact_message_detail",
            kwargs={"pk": self.object.pk},
        )


class ContactMessageDeleteView(StaffRequiredMixin, DeleteView):
    model = ContactMessage
    template_name = "core/contact_message_confirm_delete.html"
    success_url = reverse_lazy("core:contact_message_list")
```

8.4. URLs

core/urls.py

```
from django.urls import path

from . import views

app_name = "core"

urlpatterns = [
```

```
    path("", views.home, name="home"),
    path("about/", views.about, name="about"),

    path(
        "contact/",
        views.ContactMessageCreateView.as_view(),
        name="contact",
    ),
    path(
        "contact/messages/",
        views.ContactMessageListView.as_view(),
        name="contact_message_list",
    ),
    path(
        "contact/messages/<int:pk>/",
        views.ContactMessageDetailView.as_view(),
        name="contact_message_detail",
    ),
    path(
        "contact/messages/<int:pk>/edit/",
        views.ContactMessageUpdateView.as_view(),
        name="contact_message_update",
    ),
    path(
        "contact/messages/<int:pk>/delete/",
        views.ContactMessageDeleteView.as_view(),
        name="contact_message_delete",
    ),
]
```

8.5. Templates

- `templates/core/contact_form.html`
- `templates/core/contact_message_list.html`
- `templates/core/contact_message_detail.html`
- `templates/core/contact_message_form.html`
- `templates/core/contact_message_confirm_delete.html`

8.6. Tests

```
class ContactMessageViewTests(TestCase):

    def setUp(self):
        self.staff = User.objects.create_user(
            username="staff",
            email="staff@example.com",
            password="TestPassword123!",
            is_staff=True,
        )
```

```
    def test_contact_form_is_public(self):
        response = self.client.get(reverse("core:contact"))
        self.assertEqual(response.status_code, 200)

    def test_anyone_can_send_message(self):
        response = self.client.post(
            reverse("core:contact"),
            {
                "name": "Laura",
                "email": "laura@example.com",
                "subject": "Consulta",
                "message": "Hola",
            },
        )
        self.assertEqual(response.status_code, 302)
        self.assertEqual(ContactMessage.objects.count(), 1)

    def test_staff_can_see_messages(self):
        self.client.force_login(self.staff)
        response = self.client.get(reverse("core:contact_message_list"))
        self.assertEqual(response.status_code, 200)

    def test_non_staff_cannot_see_messages(self):
        user = User.objects.create_user(
            username="alumno",
            email="alumno@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.client.force_login(user)
        response = self.client.get(reverse("core:contact_message_list"))
        self.assertEqual(response.status_code, 403)
```

---

9. CRUD de `accounts` — `User`

9.1. Formulario

```
accounts/forms.py
```

Ya existe `RegisterForm` y `ProfileForm`. Añadimos un formulario para el perfil público editable (opcional) y un formulario de filtrado para el listado.

```
class UserSearchForm(forms.Form):
    q = forms.CharField(
        required=False,
        label="Buscar",
        widget=forms.TextInput(
            attrs={"placeholder": "Buscar por nombre o email"}
```

```
        ),
    )
```

9.2. Vistas

accounts/views.py

Añadir:

```
from django.contrib.auth import get_user_model
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    UserPassesTestMixin,
)
from django.views.generic import DetailView, ListView

User = get_user_model()


class UserPublicProfileView(DetailView):
    model = User
    template_name = "accounts/public_profile.html"
    context_object_name = "profile_user"
    slug_field = "username"
    slug_url_kwarg = "username"

    def get_queryset(self):
        return User.objects.filter(is_active=True)


class StaffRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):

    def test_func(self):
        return self.request.user.is_staff


class UserListView(StaffRequiredMixin, ListView):
    model = User
    template_name = "accounts/user_list.html"
    context_object_name = "users"
    paginate_by = 20

    def get_queryset(self):
        queryset = User.objects.all().order_by("-created_at")
        q = self.request.GET.get("q")
        if q:
            queryset = queryset.filter(
                models.Q(username__icontains=q)
                | models.Q(email__icontains=q)
            )
        return queryset
```

9.3. URLs

```
accounts/urls.py
```

Añadir:

```
path(
    "users/",
    views.UserListView.as_view(),
    name="user_list",
),
path(
    "users/<str:username>/",
    views.UserPublicProfileView.as_view(),
    name="public_profile",
),
```

9.4. Templates

- `templates/accounts/public_profile.html`
- `templates/accounts/user_list.html`

9.5. Tests

```
class UserProfileViewTests(TestCase):

    def setUp(self):
        self.user = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )

    def test_public_profile_is_visible(self):
        response = self.client.get(
            reverse("accounts:public_profile", kwargs={"username": "laura"})
        )
        self.assertEqual(response.status_code, 200)

    def test_user_list_requires_staff(self):
        response = self.client.get(reverse("accounts:user_list"))
        self.assertNotEqual(response.status_code, 200)

    def test_staff_can_see_user_list(self):
        self.user.is_staff = True
        self.user.save()
        self.client.force_login(self.user)
```

```
        response = self.client.get(reverse("accounts:user_list"))
        self.assertEqual(response.status_code, 200)
```

---

10. CRUD de `academies` — `Academy` y `Teacher`

10.1. Formularios

academies/forms.py

```
from django import forms

from .models import Academy, Teacher


class AcademyForm(forms.ModelForm):

    class Meta:
        model = Academy
        fields = [
            "name",
            "description",
            "city",
            "address",
            "website",
            "phone",
            "email",
            "logo",
        ]


class TeacherForm(forms.ModelForm):

    class Meta:
        model = Teacher
        fields = [
            "first_name",
            "last_name",
            "email",
            "phone",
            "specialty",
            "bio",
            "photo",
        ]

    def __init__(self, *args, **kwargs):
        self.academy = kwargs.pop("academy", None)
        super().__init__(*args, **kwargs)

    def save(self, commit=True):
```

```
        instance = super().save(commit=False)
        if self.academy and not instance.academy_id:
            instance.academy = self.academy
        if commit:
            instance.save()
        return instance
```

10.2. Vistas

academies/views.py

```
import logging

from django.contrib import messages
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    UserPassesTestMixin,
)
from django.urls import reverse_lazy
from django.views.generic import (
    CreateView,
    DeleteView,
    DetailView,
    ListView,
    UpdateView,
)

from .forms import AcademyForm, TeacherForm
from .models import Academy, Teacher

logger = logging.getLogger(__name__)


class AcademyListView(ListView):
    model = Academy
    template_name = "academies/academy_list.html"
    context_object_name = "academies"
    paginate_by = 12

    def get_queryset(self):
        return (
            Academy.objects
            .filter(is_active=True)
            .select_related("owner")
        )


class AcademyDetailView(DetailView):
    model = Academy
    template_name = "academies/academy_detail.html"
    context_object_name = "academy"
```

```
    def get_queryset(self):
        return (
            Academy.objects
            .select_related("owner")
            .prefetch_related("teachers", "courses")
        )

    def get_object(self, queryset=None):
        queryset = queryset or self.get_queryset()
        return queryset.get(slug=self.kwargs["slug"])


class AcademyCreateView(LoginRequiredMixin, UserPassesTestMixin, CreateView):
    model = Academy
    form_class = AcademyForm
    template_name = "academies/academy_form.html"

    def test_func(self):
        return self.request.user.is_academy

    def form_valid(self, form):
        form.instance.owner = self.request.user
        response = super().form_valid(form)
        messages.success(self.request, "Academia creada correctamente.")
        logger.info("Academy created: id=%s user=%s", self.object.pk, self.request.user.pk)
        return response


class AcademyOwnerMixin(LoginRequiredMixin, UserPassesTestMixin):

    def test_func(self):
        academy = self.get_object()
        return academy.owner == self.request.user


class AcademyUpdateView(AcademyOwnerMixin, UpdateView):
    model = Academy
    form_class = AcademyForm
    template_name = "academies/academy_form.html"

    def form_valid(self, form):
        response = super().form_valid(form)
        messages.success(self.request, "Academia actualizada correctamente.")
        return response


class AcademyDeleteView(AcademyOwnerMixin, DeleteView):
    model = Academy
    template_name = "academies/academy_confirm_delete.html"
    success_url = reverse_lazy("academies:list")
```

```
class TeacherListView(ListView):
    model = Teacher
    template_name = "academies/teacher_list.html"
    context_object_name = "teachers"
    paginate_by = 12

    def get_queryset(self):
        return Teacher.objects.filter(is_active=True).select_related("academy")


class TeacherDetailView(DetailView):
    model = Teacher
    template_name = "academies/teacher_detail.html"
    context_object_name = "teacher"

    def get_queryset(self):
        return Teacher.objects.select_related("academy")


class TeacherCreateView(LoginRequiredMixin, UserPassesTestMixin, CreateView):
    model = Teacher
    form_class = TeacherForm
    template_name = "academies/teacher_form.html"

    def test_func(self):
        return self.request.user.is_academy

    def get_academy(self):
        academy_pk = self.kwargs.get("academy_pk")
        return Academy.objects.get(pk=academy_pk)

    def get_form_kwargs(self):
        kwargs = super().get_form_kwargs()
        kwargs["academy"] = self.get_academy()
        return kwargs

    def test_func(self):
        academy = self.get_academy()
        return (
            self.request.user.is_academy
            and academy.owner == self.request.user
        )

    def form_valid(self, form):
        response = super().form_valid(form)
        messages.success(self.request, "Profesor creado correctamente.")
        return response

    def get_success_url(self):
        return reverse_lazy(
            "academies:detail",
            kwargs={"slug": self.object.academy.slug},
        )
```

```
class TeacherOwnerMixin(LoginRequiredMixin, UserPassesTestMixin):

    def test_func(self):
        teacher = self.get_object()
        return teacher.academy.owner == self.request.user


class TeacherUpdateView(TeacherOwnerMixin, UpdateView):
    model = Teacher
    form_class = TeacherForm
    template_name = "academies/teacher_form.html"

    def get_form_kwargs(self):
        kwargs = super().get_form_kwargs()
        kwargs["academy"] = self.object.academy
        return kwargs


class TeacherDeleteView(TeacherOwnerMixin, DeleteView):
    model = Teacher
    template_name = "academies/teacher_confirm_delete.html"

    def get_success_url(self):
        return reverse_lazy(
            "academies:detail",
            kwargs={"slug": self.object.academy.slug},
        )
```

10.3. URLs

academies/urls.py

```
from django.urls import path

from . import views

app_name = "academies"

urlpatterns = [
    path("", views.AcademyListView.as_view(), name="list"),
    path("create/", views.AcademyCreateView.as_view(), name="create"),
    path("<slug:slug>/", views.AcademyDetailView.as_view(), name="detail"),
    path("<slug:slug>/edit/", views.AcademyUpdateView.as_view(), name="update"),
    path("<slug:slug>/delete/", views.AcademyDeleteView.as_view(), name="delete"),

    path("teachers/", views.TeacherListView.as_view(), name="teacher_list"),
    path(
        "teachers/create/<int:academy_pk>/",
        views.TeacherCreateView.as_view(),
```

```
        name="teacher_create",
    ),
    path(
        "teachers/<int:pk>/",
        views.TeacherDetailView.as_view(),
        name="teacher_detail",
    ),
    path(
        "teachers/<int:pk>/edit/",
        views.TeacherUpdateView.as_view(),
        name="teacher_update",
    ),
    path(
        "teachers/<int:pk>/delete/",
        views.TeacherDeleteView.as_view(),
        name="teacher_delete",
    ),
]
```

10.4. Templates

- templates/academies/academy_list.html
- templates/academies/academy_detail.html
- templates/academies/academy_form.html
- templates/academies/academy_confirm_delete.html
- templates/academies/teacher_list.html
- templates/academies/teacher_detail.html
- templates/academies/teacher_form.html
- templates/academies/teacher_confirm_delete.html

10.5. Tests

```
class AcademyViewTests(TestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.other = User.objects.create_user(
            username="codeacademy",
            email="info@codeacademy.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
```

```
            name="English Now",
            slug="english-now",
        )

    def test_academy_list_is_public(self):
        response = self.client.get(reverse("academies:list"))
        self.assertEqual(response.status_code, 200)

    def test_academy_detail_is_public(self):
        response = self.client.get(
            reverse("academies:detail", kwargs={"slug": "english-now"})
        )
        self.assertEqual(response.status_code, 200)

    def test_create_requires_login(self):
        response = self.client.get(reverse("academies:create"))
        self.assertEqual(response.status_code, 302)

    def test_other_user_cannot_edit(self):
        self.client.force_login(self.other)
        response = self.client.get(
            reverse("academies:update", kwargs={"slug": "english-now"})
        )
        self.assertEqual(response.status_code, 403)

    def test_owner_can_edit(self):
        self.client.force_login(self.owner)
        response = self.client.get(
            reverse("academies:update", kwargs={"slug": "english-now"})
        )
        self.assertEqual(response.status_code, 200)
```

---

11. CRUD de `courses`

11.1. Formularios

courses/forms.py

```
from django import forms

from .models import Category, Convocatoria, Course, CourseImage


class CourseForm(forms.ModelForm):

    class Meta:
        model = Course
        fields = [
            "academy",
```

```
            "category",
            "teacher",
            "title",
            "description",
            "objectives",
            "competences",
            "requirements",
            "level",
            "modality",
            "enrollment_mode",
            "requires_level_test",
            "is_negotiable",
            "duration_hours",
        ]

    def __init__(self, *args, **kwargs):
        user = kwargs.pop("user", None)
        super().__init__(*args, **kwargs)
        if user is not None:
            self.fields["academy"].queryset = user.academies.filter(is_active=True)
            self.fields["teacher"].queryset = self.fields["teacher"].queryset.filter(
                academy__owner=user, is_active=True
            )


class CategoryForm(forms.ModelForm):

    class Meta:
        model = Category
        fields = ["name", "description", "icon"]


class ConvocatoriaForm(forms.ModelForm):

    class Meta:
        model = Convocatoria
        fields = [
            "name",
            "start_date",
            "enrollment_deadline",
            "schedule",
            "modality",
            "location",
            "price",
            "deposit",
            "total_places",
            "is_active",
        ]


class CourseImageForm(forms.ModelForm):

    class Meta:
```

```
        model = CourseImage
        fields = ["image", "alt_text", "is_primary", "position"]
```

11.2. Vistas

courses/views.py

```
import logging

from django.contrib import messages
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    UserPassesTestMixin,
)
from django.urls import reverse_lazy
from django.views.generic import (
    CreateView,
    DeleteView,
    DetailView,
    ListView,
    UpdateView,
)

from .forms import (
    CategoryForm,
    ConvocatoriaForm,
    CourseForm,
    CourseImageForm,
)
from .models import (
    Category,
    Convocatoria,
    Course,
    CourseImage,
    CourseStatus,
)

logger = logging.getLogger(__name__)


class CourseListView(ListView):
    model = Course
    template_name = "courses/course_list.html"
    context_object_name = "courses"
    paginate_by = 12

    def get_queryset(self):
        return (
            Course.objects
            .filter(status=CourseStatus.PUBLISHED)
            .select_related("academy", "category", "teacher")
```

```
        )


class CourseDetailView(DetailView):
    model = Course
    template_name = "courses/course_detail.html"
    context_object_name = "course"

    def get_queryset(self):
        return (
            Course.objects
            .select_related("academy", "category", "teacher")
            .prefetch_related("images", "convocatorias")
        )

    def get_object(self, queryset=None):
        queryset = queryset or self.get_queryset()
        return queryset.get(pk=self.kwargs["pk"], slug=self.kwargs["slug"])


class CourseCreateView(LoginRequiredMixin, UserPassesTestMixin, CreateView):
    model = Course
    form_class = CourseForm
    template_name = "courses/course_form.html"

    def test_func(self):
        return self.request.user.is_academy and self.request.user.academies.exists()

    def get_form_kwargs(self):
        kwargs = super().get_form_kwargs()
        kwargs["user"] = self.request.user
        return kwargs

    def form_valid(self, form):
        form.instance.status = CourseStatus.PUBLISHED
        response = super().form_valid(form)
        messages.success(self.request, "Curso publicado correctamente.")
        logger.info("Course created: id=%s user=%s", self.object.pk, self.request.user.pk)
        return response


class CourseOwnerMixin(LoginRequiredMixin, UserPassesTestMixin):

    def test_func(self):
        course = self.get_object()
        return course.academy.owner == self.request.user


class CourseUpdateView(CourseOwnerMixin, UpdateView):
    model = Course
    form_class = CourseForm
    template_name = "courses/course_form.html"
```

```
    def get_form_kwargs(self):
        kwargs = super().get_form_kwargs()
        kwargs["user"] = self.request.user
        return kwargs

    def form_valid(self, form):
        response = super().form_valid(form)
        messages.success(self.request, "Curso actualizado correctamente.")
        return response


class CourseDeleteView(CourseOwnerMixin, DeleteView):
    model = Course
    template_name = "courses/course_confirm_delete.html"
    success_url = reverse_lazy("courses:list")


class MyCourseListView(LoginRequiredMixin, ListView):
    model = Course
    template_name = "courses/my_course_list.html"
    context_object_name = "courses"

    def get_queryset(self):
        return (
            Course.objects
            .filter(academy__owner=self.request.user)
            .select_related("academy", "category", "teacher")
        )


class CategoryListView(ListView):
    model = Category
    template_name = "courses/category_list.html"
    context_object_name = "categories"

    def get_queryset(self):
        return Category.objects.filter(is_active=True)


class CategoryDetailView(DetailView):
    model = Category
    template_name = "courses/category_detail.html"
    context_object_name = "category"

    def get_object(self, queryset=None):
        queryset = queryset or Category.objects.all()
        return queryset.get(slug=self.kwargs["slug"])


class StaffRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):

    def test_func(self):
        return self.request.user.is_staff
```

```
class CategoryCreateView(StaffRequiredMixin, CreateView):
    model = Category
    form_class = CategoryForm
    template_name = "courses/category_form.html"
    success_url = reverse_lazy("courses:category_list")


class CategoryUpdateView(StaffRequiredMixin, UpdateView):
    model = Category
    form_class = CategoryForm
    template_name = "courses/category_form.html"
    success_url = reverse_lazy("courses:category_list")


class CategoryDeleteView(StaffRequiredMixin, DeleteView):
    model = Category
    template_name = "courses/category_confirm_delete.html"
    success_url = reverse_lazy("courses:category_list")


class ConvocatoriaCreateView(CourseOwnerMixin, CreateView):
    model = Convocatoria
    form_class = ConvocatoriaForm
    template_name = "courses/convocatoria_form.html"

    def get_course(self):
        return Course.objects.get(pk=self.kwargs["course_pk"])

    def test_func(self):
        course = self.get_course()
        return course.academy.owner == self.request.user

    def form_valid(self, form):
        form.instance.course = self.get_course()
        return super().form_valid(form)

    def get_success_url(self):
        return self.object.course.get_absolute_url()


class ConvocatoriaOwnerMixin(LoginRequiredMixin, UserPassesTestMixin):

    def test_func(self):
        convocatoria = self.get_object()
        return convocatoria.course.academy.owner == self.request.user


class ConvocatoriaUpdateView(ConvocatoriaOwnerMixin, UpdateView):
    model = Convocatoria
    form_class = ConvocatoriaForm
    template_name = "courses/convocatoria_form.html"
```

```
    def get_success_url(self):
        return self.object.course.get_absolute_url()


class ConvocatoriaDeleteView(ConvocatoriaOwnerMixin, DeleteView):
    model = Convocatoria
    template_name = "courses/convocatoria_confirm_delete.html"

    def get_success_url(self):
        return self.object.course.get_absolute_url()


class CourseImageCreateView(CourseOwnerMixin, CreateView):
    model = CourseImage
    form_class = CourseImageForm
    template_name = "courses/course_image_form.html"

    def get_course(self):
        return Course.objects.get(pk=self.kwargs["course_pk"])

    def test_func(self):
        course = self.get_course()
        return course.academy.owner == self.request.user

    def form_valid(self, form):
        form.instance.course = self.get_course()
        return super().form_valid(form)

    def get_success_url(self):
        return self.object.course.get_absolute_url()


class CourseImageOwnerMixin(LoginRequiredMixin, UserPassesTestMixin):

    def test_func(self):
        image = self.get_object()
        return image.course.academy.owner == self.request.user


class CourseImageUpdateView(CourseImageOwnerMixin, UpdateView):
    model = CourseImage
    form_class = CourseImageForm
    template_name = "courses/course_image_form.html"

    def get_success_url(self):
        return self.object.course.get_absolute_url()


class CourseImageDeleteView(CourseImageOwnerMixin, DeleteView):
    model = CourseImage
    template_name = "courses/course_image_confirm_delete.html"
```

```
    def get_success_url(self):
        return self.object.course.get_absolute_url()
```

11.3. URLs

courses/urls.py

```
from django.urls import path

from . import views

app_name = "courses"

urlpatterns = [
    path("", views.CourseListView.as_view(), name="list"),
    path("create/", views.CourseCreateView.as_view(), name="create"),
    path("mine/", views.MyCourseListView.as_view(), name="mine"),

    path(
        "categories/",
        views.CategoryListView.as_view(),
        name="category_list",
    ),
    path(
        "categories/create/",
        views.CategoryCreateView.as_view(),
        name="category_create",
    ),
    path(
        "categories/<slug:slug>/",
        views.CategoryDetailView.as_view(),
        name="category_detail",
    ),
    path(
        "categories/<slug:slug>/edit/",
        views.CategoryUpdateView.as_view(),
        name="category_update",
    ),
    path(
        "categories/<slug:slug>/delete/",
        views.CategoryDeleteView.as_view(),
        name="category_delete",
    ),

    path(
        "<int:pk>/<slug:slug>/",
        views.CourseDetailView.as_view(),
        name="detail",
    ),
    path(
        "<int:pk>/edit/",
```

```
        views.CourseUpdateView.as_view(),
        name="update",
    ),
    path(
        "<int:pk>/delete/",
        views.CourseDeleteView.as_view(),
        name="delete",
    ),

    path(
        "<int:course_pk>/convocatorias/create/",
        views.ConvocatoriaCreateView.as_view(),
        name="convocatoria_create",
    ),
    path(
        "convocatorias/<int:pk>/edit/",
        views.ConvocatoriaUpdateView.as_view(),
        name="convocatoria_update",
    ),
    path(
        "convocatorias/<int:pk>/delete/",
        views.ConvocatoriaDeleteView.as_view(),
        name="convocatoria_delete",
    ),

    path(
        "<int:course_pk>/images/create/",
        views.CourseImageCreateView.as_view(),
        name="course_image_create",
    ),
    path(
        "images/<int:pk>/edit/",
        views.CourseImageUpdateView.as_view(),
        name="course_image_update",
    ),
    path(
        "images/<int:pk>/delete/",
        views.CourseImageDeleteView.as_view(),
        name="course_image_delete",
    ),
]
```

11.4. Templates

- `templates/courses/course_list.html`
- `templates/courses/course_detail.html`
- `templates/courses/course_form.html`
- `templates/courses/course_confirm_delete.html`
- `templates/courses/my_course_list.html`
- `templates/courses/category_list.html`
- `templates/courses/category_detail.html`

- templates/courses/category_form.html
- templates/courses/category_confirm_delete.html
- templates/courses/convocatoria_form.html
- templates/courses/convocatoria_confirm_delete.html
- templates/courses/course_image_form.html
- templates/courses/course_image_confirm_delete.html

11.5. Tests

```
class CourseViewTests(TestCase):

    def setUp(self):
        self.academy_user = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.other_academy_user = User.objects.create_user(
            username="codeacademy",
            email="info@codeacademy.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.academy = Academy.objects.create(
            owner=self.academy_user,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(
            name="Idiomas",
            slug="idiomas",
        )
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="Curso B2",
            status=CourseStatus.PUBLISHED,
        )

    def test_list_is_public(self):
        response = self.client.get(reverse("courses:list"))
        self.assertEqual(response.status_code, 200)

    def test_detail_is_public(self):
        response = self.client.get(
            reverse("courses:detail", kwargs={"pk": self.course.pk, "slug": "ingles-b2"})
        )
        self.assertEqual(response.status_code, 200)
```

```
    def test_only_published_in_list(self):
        draft = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Curso oculto",
            slug="curso-oculto",
            description="...",
            status=CourseStatus.DRAFT,
        )
        response = self.client.get(reverse("courses:list"))
        self.assertContains(response, "Inglés B2")
        self.assertNotContains(response, "Curso oculto")

    def test_create_requires_login(self):
        response = self.client.get(reverse("courses:create"))
        self.assertEqual(response.status_code, 302)

    def test_other_user_cannot_edit(self):
        self.client.force_login(self.other_academy_user)
        response = self.client.get(
            reverse("courses:update", kwargs={"pk": self.course.pk})
        )
        self.assertEqual(response.status_code, 403)

    def test_owner_can_edit(self):
        self.client.force_login(self.academy_user)
        response = self.client.get(
            reverse("courses:update", kwargs={"pk": self.course.pk})
        )
        self.assertEqual(response.status_code, 200)
```

---

12. CRUD de `enrollments`

12.1. Formularios

enrollments/forms.py

```
from django import forms

from .models import Enrollment, EnrollmentAnswer, EnrollmentDocument


class EnrollmentForm(forms.ModelForm):

    class Meta:
        model = Enrollment
        fields = []
```

```
    def __init__(self, *args, **kwargs):
        self.student = kwargs.pop("student", None)
        self.convocatoria = kwargs.pop("convocatoria", None)
        super().__init__(*args, **kwargs)

    def save(self, commit=True):
        instance = super().save(commit=False)
        if self.student and not instance.student_id:
            instance.student = self.student
        if self.convocatoria and not instance.convocatoria_id:
            instance.convocatoria = self.convocatoria
        if commit:
            instance.save()
        return instance


class EnrollmentAnswerForm(forms.ModelForm):

    class Meta:
        model = EnrollmentAnswer
        fields = ["question_key", "question_text", "answer_text"]


class EnrollmentDocumentForm(forms.ModelForm):

    class Meta:
        model = EnrollmentDocument
        fields = ["name", "file"]
```

12.2. Vistas

enrollments/views.py

```
import logging

from django.contrib import messages
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    UserPassesTestMixin,
)
from django.shortcuts import get_object_or_404
from django.urls import reverse, reverse_lazy
from django.views.generic import (
    CreateView,
    DeleteView,
    DetailView,
    ListView,
    UpdateView,
)

from courses.models import Convocatoria
```

```
from .forms import (
    EnrollmentAnswerForm,
    EnrollmentDocumentForm,
    EnrollmentForm,
)
from .models import (
    Enrollment,
    EnrollmentAnswer,
    EnrollmentDocument,
    EnrollmentStatus,
    WaitlistEntry,
)

logger = logging.getLogger(__name__)


class EnrollmentListView(LoginRequiredMixin, ListView):
    model = Enrollment
    template_name = "enrollments/my_enrollment_list.html"
    context_object_name = "enrollments"

    def get_queryset(self):
        return (
            Enrollment.objects
            .filter(student=self.request.user)
            .select_related("convocatoria", "convocatoria__course")
            .order_by("-created_at")
        )


class EnrollmentDetailView(LoginRequiredMixin, UserPassesTestMixin, DetailView):
    model = Enrollment
    template_name = "enrollments/enrollment_detail.html"
    context_object_name = "enrollment"

    def test_func(self):
        enrollment = self.get_object()
        return (
            enrollment.student == self.request.user
            or enrollment.convocatoria.course.academy.owner == self.request.user
        )


class EnrollmentCreateView(LoginRequiredMixin, CreateView):
    model = Enrollment
    form_class = EnrollmentForm
    template_name = "enrollments/enrollment_form.html"

    def dispatch(self, request, *args, **kwargs):
        self.convocatoria = get_object_or_404(
            Convocatoria,
            pk=kwargs["convocatoria_pk"],
```

```
        )
        return super().dispatch(request, *args, **kwargs)

    def get_form_kwargs(self):
        kwargs = super().get_form_kwargs()
        kwargs["student"] = self.request.user
        kwargs["convocatoria"] = self.convocatoria
        return kwargs

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["convocatoria"] = self.convocatoria
        return context

    def form_valid(self, form):
        existing = Enrollment.objects.filter(
            student=self.request.user,
            convocatoria=self.convocatoria,
        ).exists()
        if existing:
            messages.error(
                self.request,
                "Ya tienes una inscripción en esta convocatoria.",
            )
            return self.form_invalid(form)

        if self.convocatoria.is_full:
            WaitlistEntry.objects.get_or_create(
                student=self.request.user,
                convocatoria=self.convocatoria,
                defaults={"position": WaitlistEntry.objects.filter(
                    convocatoria=self.convocatoria
                ).count() + 1},
            )
            messages.warning(
                self.request,
                "La convocatoria está llena. Te hemos añadido a la lista de espera.",
            )
            return super().form_valid(form)

        response = super().form_valid(form)
        messages.success(self.request, "Inscripción enviada correctamente.")
        logger.info(
            "Enrollment created: id=%s student=%s",
            self.object.pk,
            self.request.user.pk,
        )
        return response

    def get_success_url(self):
        return reverse("enrollments:detail", kwargs={"pk": self.object.pk})
```

```
class EnrollmentCancelView(LoginRequiredMixin, UserPassesTestMixin, UpdateView):
    model = Enrollment
    fields = []
    template_name = "enrollments/enrollment_confirm_cancel.html"

    def test_func(self):
        enrollment = self.get_object()
        return enrollment.student == self.request.user

    def form_valid(self, form):
        enrollment = form.save(commit=False)
        enrollment.status = EnrollmentStatus.CANCELLED
        enrollment.save()
        messages.success(self.request, "Inscripción cancelada.")
        return super().form_valid(form)

    def get_success_url(self):
        return reverse("enrollments:list")


class AcademyEnrollmentListView(LoginRequiredMixin, UserPassesTestMixin, ListView):
    model = Enrollment
    template_name = "enrollments/academy_enrollment_list.html"
    context_object_name = "enrollments"

    def test_func(self):
        return self.request.user.is_academy

    def get_queryset(self):
        return (
            Enrollment.objects
            .filter(
                convocatoria__course__academy__owner=self.request.user
            )
            .select_related("student", "convocatoria", "convocatoria__course")
            .order_by("-created_at")
        )


class EnrollmentApproveView(LoginRequiredMixin, UserPassesTestMixin, UpdateView):
    model = Enrollment
    fields = []
    template_name = "enrollments/enrollment_confirm_approve.html"

    def test_func(self):
        enrollment = self.get_object()
        return enrollment.convocatoria.course.academy.owner == self.request.user

    def form_valid(self, form):
        enrollment = form.save(commit=False)
        enrollment.status = EnrollmentStatus.APPROVED
        enrollment.save()
        messages.success(self.request, "Inscripción aprobada.")
```

```
        return super().form_valid(form)

    def get_success_url(self):
        return reverse("enrollments:academy_list")


class EnrollmentRejectView(LoginRequiredMixin, UserPassesTestMixin, UpdateView):
    model = Enrollment
    fields = []
    template_name = "enrollments/enrollment_confirm_reject.html"

    def test_func(self):
        enrollment = self.get_object()
        return enrollment.convocatoria.course.academy.owner == self.request.user

    def form_valid(self, form):
        enrollment = form.save(commit=False)
        enrollment.status = EnrollmentStatus.REJECTED
        enrollment.save()
        messages.success(self.request, "Inscripción rechazada.")
        return super().form_valid(form)

    def get_success_url(self):
        return reverse("enrollments:academy_list")


class EnrollmentEnrollView(LoginRequiredMixin, UserPassesTestMixin, UpdateView):
    model = Enrollment
    fields = []
    template_name = "enrollments/enrollment_confirm_enroll.html"

    def test_func(self):
        enrollment = self.get_object()
        return enrollment.convocatoria.course.academy.owner == self.request.user

    def form_valid(self, form):
        enrollment = form.save(commit=False)
        enrollment.status = EnrollmentStatus.ENROLLED
        enrollment.save()
        messages.success(self.request, "Alumno matriculado.")
        return super().form_valid(form)

    def get_success_url(self):
        return reverse("enrollments:academy_list")


class EnrollmentAnswerCreateView(LoginRequiredMixin, UserPassesTestMixin, CreateView):
    model = EnrollmentAnswer
    form_class = EnrollmentAnswerForm
    template_name = "enrollments/answer_form.html"

    def get_enrollment(self):
        return get_object_or_404(Enrollment, pk=self.kwargs["enrollment_pk"])
```

```
    def test_func(self):
        enrollment = self.get_enrollment()
        return enrollment.student == self.request.user

    def form_valid(self, form):
        form.instance.enrollment = self.get_enrollment()
        return super().form_valid(form)

    def get_success_url(self):
        return reverse(
            "enrollments:detail",
            kwargs={"pk": self.kwargs["enrollment_pk"]},
        )


class EnrollmentDocumentCreateView(LoginRequiredMixin, UserPassesTestMixin, CreateView):
    model = EnrollmentDocument
    form_class = EnrollmentDocumentForm
    template_name = "enrollments/document_form.html"

    def get_enrollment(self):
        return get_object_or_404(Enrollment, pk=self.kwargs["enrollment_pk"])

    def test_func(self):
        enrollment = self.get_enrollment()
        return enrollment.student == self.request.user

    def form_valid(self, form):
        form.instance.enrollment = self.get_enrollment()
        return super().form_valid(form)

    def get_success_url(self):
        return reverse(
            "enrollments:detail",
            kwargs={"pk": self.kwargs["enrollment_pk"]},
        )


class EnrollmentDocumentVerifyView(LoginRequiredMixin, UserPassesTestMixin, UpdateView):
    model = EnrollmentDocument
    fields = []
    template_name = "enrollments/document_confirm_verify.html"

    def test_func(self):
        document = self.get_object()
        return (
            document.enrollment.convocatoria.course.academy.owner
            == self.request.user
        )

    def form_valid(self, form):
        document = form.save(commit=False)
```

```
        document.is_verified = True
        document.save()
        messages.success(self.request, "Documento verificado.")
        return super().form_valid(form)

    def get_success_url(self):
        return reverse(
            "enrollments:detail",
            kwargs={"pk": self.object.enrollment.pk},
        )
```

12.3. URLs

enrollments/urls.py

```
from django.urls import path

from . import views

app_name = "enrollments"

urlpatterns = [
    path("", views.EnrollmentListView.as_view(), name="list"),
    path(
        "create/<int:convocatoria_pk>/",
        views.EnrollmentCreateView.as_view(),
        name="create",
    ),
    path(
        "<int:pk>/",
        views.EnrollmentDetailView.as_view(),
        name="detail",
    ),
    path(
        "<int:pk>/cancel/",
        views.EnrollmentCancelView.as_view(),
        name="cancel",
    ),
    path(
        "<int:pk>/approve/",
        views.EnrollmentApproveView.as_view(),
        name="approve",
    ),
    path(
        "<int:pk>/reject/",
        views.EnrollmentRejectView.as_view(),
        name="reject",
    ),
    path(
        "<int:pk>/enroll/",
        views.EnrollmentEnrollView.as_view(),
```

```
        name="enroll",
    ),
    path(
        "academy/",
        views.AcademyEnrollmentListView.as_view(),
        name="academy_list",
    ),
    path(
        "<int:enrollment_pk>/answers/create/",
        views.EnrollmentAnswerCreateView.as_view(),
        name="answer_create",
    ),
    path(
        "<int:enrollment_pk>/documents/create/",
        views.EnrollmentDocumentCreateView.as_view(),
        name="document_create",
    ),
    path(
        "documents/<int:pk>/verify/",
        views.EnrollmentDocumentVerifyView.as_view(),
        name="document_verify",
    ),
]
```

12.4. Templates

- templates/enrollments/my_enrollment_list.html
- templates/enrollments/enrollment_detail.html
- templates/enrollments/enrollment_form.html
- templates/enrollments/enrollment_confirm_cancel.html
- templates/enrollments/academy_enrollment_list.html
- templates/enrollments/enrollment_confirm_approve.html
- templates/enrollments/enrollment_confirm_reject.html
- templates/enrollments/enrollment_confirm_enroll.html
- templates/enrollments/answer_form.html
- templates/enrollments/document_form.html
- templates/enrollments/document_confirm_verify.html

12.5. Tests

```
class EnrollmentViewTests(TestCase):

    def setUp(self):
        self.academy_user = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
```

```
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.academy_user,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(
            name="Idiomas",
            slug="idiomas",
        )
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
        )
        self.convocatoria = Convocatoria.objects.create(
            course=self.course,
            name="Verano",
            start_date=timezone.now().date() + timedelta(days=30),
            price=Decimal("480.00"),
            total_places=15,
        )

    def test_student_can_enroll(self):
        self.client.force_login(self.student)
        response = self.client.post(
            reverse("enrollments:create", kwargs={"convocatoria_pk": self.convocatoria.pk}),
            {},
        )
        self.assertEqual(response.status_code, 302)
        self.assertEqual(Enrollment.objects.count(), 1)

    def test_enrollment_requires_login(self):
        response = self.client.post(
            reverse("enrollments:create", kwargs={"convocatoria_pk": self.convocatoria.pk}),
            {},
        )
        self.assertEqual(response.status_code, 302)

    def test_other_user_cannot_see_enrollment(self):
        other_student = User.objects.create_user(
            username="carlos",
            email="carlos@example.com",
            password="TestPassword123!",
            is_student=True,
        )
```

```
        self.client.force_login(self.student)
        self.client.post(
            reverse("enrollments:create", kwargs={"convocatoria_pk": self.convocatoria.pk}),
            {},
        )
        enrollment = Enrollment.objects.first()
        self.client.force_login(other_student)
        response = self.client.get(reverse("enrollments:detail", kwargs={"pk": enrollment.pk}
        self.assertEqual(response.status_code, 403)

    def test_academy_can_approve(self):
        self.client.force_login(self.student)
        self.client.post(
            reverse("enrollments:create", kwargs={"convocatoria_pk": self.convocatoria.pk}),
            {},
        )
        enrollment = Enrollment.objects.first()
        self.client.force_login(self.academy_user)
        response = self.client.post(
            reverse("enrollments:approve", kwargs={"pk": enrollment.pk}),
            {},
        )
        enrollment.refresh_from_db()
        self.assertEqual(enrollment.status, EnrollmentStatus.APPROVED)
```

---

13. URLs finales por app

core

```
/                          → home
/about/                    → about
/contact/                  → contact
/contact/messages/         → contact_message_list (staff)
/contact/messages/<pk>/    → contact_message_detail (staff)
/contact/messages/<pk>/edit/   → contact_message_update (staff)
/contact/messages/<pk>/delete/ → contact_message_delete (staff)
```

accounts

```
/accounts/register/        → register
/accounts/login/           → login
/accounts/logout/          → logout
/accounts/profile/         → profile
/accounts/profile/edit/    → profile_edit
/accounts/users/           → user_list (staff)
/accounts/users/<username>/ → public_profile
```

academies

```
/academies/                        → list
/academies/create/                 → create
/academies/<slug>/                 → detail
/academies/<slug>/edit/            → update
/academies/<slug>/delete/          → delete
/academies/teachers/               → teacher_list
/academies/teachers/create/<pk>/   → teacher_create
/academies/teachers/<pk>/          → teacher_detail
/academies/teachers/<pk>/edit/     → teacher_update
/academies/teachers/<pk>/delete/   → teacher_delete
```

courses

```
/courses/                                → list
/courses/create/                         → create
/courses/mine/                           → mine
/courses/categories/                     → category_list
/courses/categories/create/              → category_create (staff)
/courses/categories/<slug>/              → category_detail
/courses/categories/<slug>/edit/         → category_update (staff)
/courses/categories/<slug>/delete/       → category_delete (staff)
/courses/<pk>/<slug>/                    → detail
/courses/<pk>/edit/                      → update
/courses/<pk>/delete/                    → delete
/courses/<pk>/convocatorias/create/      → convocatoria_create
/courses/convocatorias/<pk>/edit/        → convocatoria_update
/courses/convocatorias/<pk>/delete/      → convocatoria_delete
/courses/<pk>/images/create/             → course_image_create
/courses/images/<pk>/edit/               → course_image_update
/courses/images/<pk>/delete/             → course_image_delete
```

enrollments

```
/enrollments/                            → list (alumno)
/enrollments/create/<pk>/                → create
/enrollments/<pk>/                       → detail
/enrollments/<pk>/cancel/                → cancel
/enrollments/<pk>/approve/               → approve (academia)
/enrollments/<pk>/reject/                → reject (academia)
/enrollments/<pk>/enroll/                → enroll (academia)
/enrollments/academy/                    → academy_list (academia)
/enrollments/<pk>/answers/create/        → answer_create
/enrollments/<pk>/documents/create/      → document_create
/enrollments/documents/<pk>/verify/      → document_verify
```

---

14. Tests transversales

Además de los tests por app, añadimos los siguientes casos críticos:

- Listados públicos accesibles sin login.
- Detalles públicos accesibles sin login.
- Crear requiere login.
- Editar requiere ser propietario.
- Eliminar requiere ser propietario.
- Tercero recibe 403 al intentar editar o eliminar.
- Staff puede ver mensajes de contacto.
- Staff no puede ver mensajes si no es staff.
- Alumno ve solo sus inscripciones.
- Academia ve solo las inscripciones de sus cursos.

Ejecutar:

```
python manage.py test
```

Resultado esperado:

```
OK
```

---

15. Checklist global de la sesión

`core`

- ContactMessageCreateView.
- ContactMessageListView.
- ContactMessageDetailView.
- ContactMessageUpdateView.
- ContactMessageDeleteView.
- Templates.
- Tests.

`accounts`

- UserPublicProfileView.
- UserListView.
- Templates.
- Tests.

`academies`

- AcademyListView.
- AcademyDetailView.
- AcademyCreateView.

- AcademyUpdateView.
- AcademyDeleteView.
- TeacherListView.
- TeacherDetailView.
- TeacherCreateView.
- TeacherUpdateView.
- TeacherDeleteView.
- Templates.
- Tests.

courses

- CourseListView.
- CourseDetailView.
- CourseCreateView.
- CourseUpdateView.
- CourseDeleteView.
- MyCourseListView.
- CategoryListView.
- CategoryDetailView.
- CategoryCreateView (staff).
- CategoryUpdateView (staff).
- CategoryDeleteView (staff).
- ConvocatoriaCreateView.
- ConvocatoriaUpdateView.
- ConvocatoriaDeleteView.
- CourseImageCreateView.
- CourseImageUpdateView.
- CourseImageDeleteView.
- Templates.
- Tests.

enrollments

- EnrollmentListView.
- EnrollmentDetailView.
- EnrollmentCreateView.
- EnrollmentCancelView.
- AcademyEnrollmentListView.
- EnrollmentApproveView.
- EnrollmentRejectView.
- EnrollmentEnrollView.
- EnrollmentAnswerCreateView.
- EnrollmentDocumentCreateView.

- EnrollmentDocumentVerifyView.
- Templates.
- Tests.

---

16. Commit recomendado

```
git status
git add .
git commit -m "feat: implement full CRUD across all apps"
```

---

17. Registro de sesión

```
PROYECTO: Academy Hub
SESIÓN: 05
FASE: Desarrollo del núcleo de la plataforma
TEMA: CRUD completo en todas las apps
FECHA:
HORA DE ENTRADA:
HORA DE SALIDA:
DURACIÓN:

OBJETIVO:
Aplicar el patrón CRUD completo (del PDF de la Sesión 5) a todas las apps
existentes de Academy Hub.

APPS CUBIERTAS:
- core
- accounts
- academies
- courses
- enrollments

FUNCIONALIDADES:
- CRUD completo por app.
- Permisos por rol y por propietario.
- Object-level authorization.
- Paginación.
- Optimización de querysets.
- Logging de acciones críticas.

TESTS:
- listados públicos
- filtrado por estado
- detalles públicos
- creación requiere login
- creación asigna propietario
- propietario edita
```

```
- tercero recibe 403
- propietario elimina
- tercero no puede eliminar
- staff ve mensajes
- alumno ve solo sus inscripciones
- academia aprueba / rechaza / matricula

COMMIT:
feat: implement full CRUD across all apps

ESTADO:
[ ] Pendiente
[ ] En progreso
[ ] Completada

PRÓXIMA SESIÓN:
Imágenes múltiples, favoritos, búsqueda, filtros y consultas ORM avanzadas.
```

---

18. Resultado final

Antes de esta sesión, Academy Hub tenía modelos y un sistema de autenticación básico.

Ahora tiene:

```
Visitante
    puede ver:
        - academias
        - profesores
        - cursos
        - categorías

Alumno
    puede:
        - inscribirse en convocatorias
        - apuntarse a lista de espera
        - subir documentos
        - ver sus inscripciones
        - cancelar sus inscripciones

Academia
    puede:
        - crear y gestionar sus academias
        - crear y gestionar sus profesores
        - crear y gestionar sus cursos
        - crear y gestionar sus convocatorias
        - crear y gestionar sus imágenes
        - aprobar / rechazar / matricular alumnos

Staff
    puede:
        - ver mensajes de contacto
```

```
        - gestionar usuarios
        - gestionar categorías
```

En la Sesión 6, Academy Hub dejará de ser únicamente un CRUD y empezará a comportarse más como una plataforma real:

- múltiples imágenes,
- favoritos,
- búsqueda por texto,
- filtros por categoría, nivel, modalidad y precio,
- paginación mejorada,
- consultas avanzadas con Q.